SHIPSCAN

PROJECT DOCUMENTATION

Automated extraction of PDF attachments (shipping bills, invoices) from Gmail

Web application · Python / Flask · Gmail IMAP

1. Executive Summary

Project Name ShipScan

Objective To automate the retrieval of PDF attachments — such as shipping bills, invoices, and customs documents — from a Gmail mailbox, replacing the slow, error-prone task of opening hundreds of emails and downloading attachments by hand.

The solution aims to:

- **Eliminate manual effort —** Eliminate the manual, repetitive task of locating and saving email attachments
- **Improve accuracy —** Pull only the documents that matter, using filename and pattern filters
- **Improve speed —** Move the search work onto the mail server so it stays fast even on large mailboxes
- **Improve traceability —** Produce a clear, color-coded record of every file retrieved
- **Stay accessible —** Let a non-technical user run the whole process from a simple web page

2. What is ShipScan?

ShipScan is a web application that connects to a Gmail mailbox, scans the messages from a chosen sender, finds the PDF attachments, and downloads the ones the user wants — automatically, in a single run.

Instead of manually opening each email and saving each file, ShipScan:

- Reads sender, date, and subject to narrow the search on the mail server
- Inspects each message's structure to discover attachment filenames without downloading the files
- Matches those filenames against the user's filters (exact names or wildcard patterns)
- Downloads only the matching PDFs, skipping anything already saved
- Builds an Excel report listing every file, its source email, and its status

Example

Without ShipScan A logistics clerk opens 300 supplier emails one by one, downloads the shipping-bill PDF from each, renames it, and files it — a task that takes hours and invites mistakes.

With ShipScan The clerk enters the sender, a date range, and a filename pattern, clicks one button, and receives all matching PDFs plus a spreadsheet log in a couple of minutes.

3. Why is ShipScan Necessary?

Current Challenges

Teams that receive shipping and trade documents by email routinely face:

- **Manual effort —** Significant time is lost opening individual emails and saving attachments by hand.

- **Human error —** Missed emails, wrong files saved, duplicate downloads, and inconsistent naming.
- **Lack of scale —** As email volume grows, manual collection becomes impractical.
- **Delays —** Documents needed for clearance or accounting are not gathered in time.
- **Poor tracking —** No simple record of which files were collected, from which email, and when.

4. Business Problem Statement

Existing Situation

Operations, logistics, and accounts teams regularly retrieve PDF documents that arrive as email attachments, including:

- Shipping bills and bills of lading
- Commercial invoices and packing lists
- Customs and clearance paperwork
- Supplier statements and remittance advices

Collecting these documents is currently a manual, repetitive process that consumes valuable working time.

Business Impact

- Increased operational cost from time spent on low-value work
- Reduced staff productivity
- Delays in downstream processes such as customs clearance and reconciliation
- Inconsistent filing and risk of missed or duplicated documents

Proposed Solution

Deploy ShipScan: a lightweight web tool that connects to the mailbox, finds the right PDFs using sender, date, subject, and filename filters, and downloads them automatically while producing an audit-ready report.

5. Solution Overview and Key Features

ShipScan offers two complementary ways to work, both built on the same fast extraction pipeline.

Mode 1 — Targeted Extract

The user supplies one or more filename filters and ShipScan automatically downloads every matching PDF. Filters can be exact names or wildcard patterns.

- Exact match, e.g. 348.pdf
- Wildcard patterns, e.g. SB_*_2024.pdf or *invoice*
- Leave the filter blank to fetch every PDF from the sender
- Files already present on disk are detected and skipped

Mode 2 — Browse and Pick

ShipScan lists every PDF from the sender, newest first, showing filename, date, subject, and size. The user ticks the files they want and saves just those.

Shared Capabilities

- **Smart search —** Search by sender email, date range (days back), and optional subject phrases (matched as OR)
- **Excel report —** A color-coded spreadsheet listing each file, its email, and a Saved / Failed / Already-exists status
- **One-click download —** In the cloud build, all results are delivered as a single ZIP download
- **Progress feedback —** Live status log shows progress as the run proceeds

6. Technology Stack Evaluation and Recommendation

Three approaches were considered for collecting these documents.

Option 1 — Manual Download (Baseline)

Staff continue to open emails and save attachments by hand. No technology is required, but the approach does not scale and offers no record of what was collected.

Option 2 — ShipScan Web App (Recommended)

Technology stack: Python 3.12, Flask, IMAPClient (Gmail IMAP), openpyxl (Excel), Gunicorn, deployed on Render.

Advantages:

- Low implementation cost and minimal infrastructure (a single web service)
- Fast even on large mailboxes thanks to server-side metadata scanning
- Flexible filename and pattern filtering
- Built-in Excel reporting for an audit trail
- Usable by non-technical staff through a simple web page

Option 3 — Cloud-Native Ingestion Pipeline

Technology stack: Serverless functions (e.g. AWS Lambda), object storage (S3), a database, and inbound email processing.

Highly scalable and fully automated, but it carries higher cost, greater technical complexity, and a longer setup time — more than current needs require.

Comparison

Criteria	Manual download	ShipScan (recommended)	Cloud pipeline
Implementation cost	None	Low	High
Setup effort	None	Low	High
Speed on large mailboxes	Very slow	Fast	Fast
Filtering flexibility	Manual only	High (wildcards)	High
Audit trail / reporting	None	Excel log	Database / dashboards

Criteria	Manual download	ShipScan (recommended)	Cloud pipeline
Infrastructure requirement	None	Minimal (1 service)	Significant
Technical complexity	Low	Low	High
Scalability	Very low	Medium	Very high
Business-user accessibility	High	High	Low
Best fit	One-off, few files	Recurring, focused use	Enterprise volumes

Final Recommendation

Recommended Solution ShipScan — the Flask + Gmail IMAP web application.

Justification:

- It directly solves the current problem of collecting PDF documents from email
- It requires minimal infrastructure and is inexpensive to run
- It is fast on large mailboxes and flexible in what it retrieves
- It produces a clear record of every file collected
- It can be operated by business users with no coding knowledge
- It leaves room to grow into a fuller pipeline later if volumes increase

7. How ShipScan Works (Architecture)

Gmail inbox → IMAP metadata scan → Filter match? → Download matched PDF part → Save + Excel → Deliver

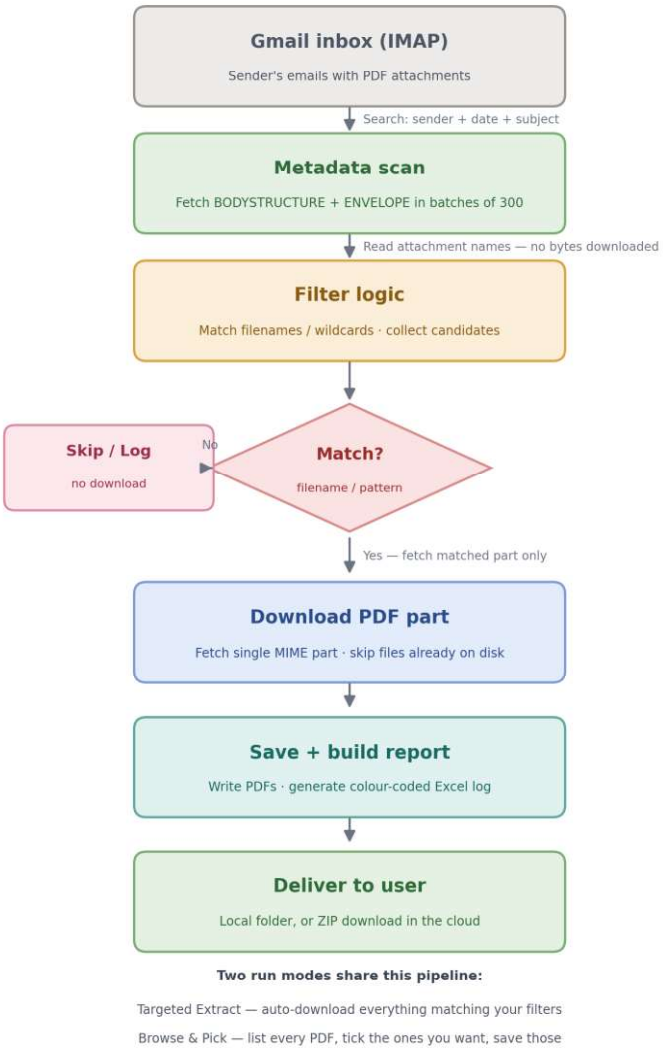

Figure 1 — ShipScan extraction pipeline

The key idea: scan metadata first, download later

Earlier approaches downloaded every matching email in full — including all attachment bytes — just to read the attachment names. On a mailbox with thousands of emails, that meant transferring hundreds of megabytes to find a handful of files.

ShipScan instead works in two phases:

- **Phase 1 — Metadata scan:** ShipScan asks the mail server only for each message's structure and envelope (tiny metadata, no attachment bytes), in batches of 300 messages per round-trip. It reads attachment filenames straight from that metadata and matches them against the filters.

- **Phase 2 — Targeted download:** ShipScan downloads the bytes only for the PDF parts that matched and are not already on disk — fetching just that one attachment, not the whole email.

> *Net effect: the "find the right files" work happens almost entirely on the server, and only the PDFs actually kept are transferred — so runs stay fast even on very large mailboxes.*

8. Security and Deployment

Security

- **App Passwords —** ShipScan uses a Gmail App Password, never the account's real password. App Passwords can be revoked at any time from the Google account settings.
- **Access gate —** The hosted site can be locked behind a password (HTTP Basic auth) so the public URL is not open to anyone.
- **Read-only access —** The mailbox is opened in read-only mode; ShipScan never alters or deletes email.

Deployment

- The app is deployed as a single web service on Render (free tier supported).
- It runs under Gunicorn with one worker and a long request timeout, since a large fetch runs on a single request.
- On the cloud build, results are delivered as a ZIP download because the hosted disk is temporary.

Limitations

- Designed for one user at a time; it is a focused personal/team tool, not a multi-tenant product.
- Hosted files are temporary — download the ZIP when a run finishes.
- Subject to Gmail's IMAP limits; best suited to small and medium volumes.
- On the free hosting tier the service sleeps after inactivity, so the first visit may take a moment to wake.

9. Conclusion

- ShipScan streamlines and automates the repetitive task of collecting PDF documents from email. By replacing manual downloading with an automated, filter-driven workflow, it improves speed, accuracy, and traceability.
- After comparing a manual baseline and a heavier cloud-native pipeline, the ShipScan web app (Flask + Gmail IMAP) was selected as the best fit for current needs: low cost, easy to run, fast on large mailboxes, and accessible to non-technical users.
- The solution meets immediate requirements while leaving a clear path to a fuller pipeline should document volumes grow.

Key Outcomes Expected

- Reduced manual effort in collecting email attachments
- Improved accuracy and consistent filing
- Faster turnaround on document-dependent processes

- A clear, audit-ready record of every file retrieved
- A scalable foundation for future growth

10. References

Deployment guide: README_DEPLOY.md (bundled with the project) — step-by-step instructions for deploying ShipScan to Render.

Hosting platform: https://render.com

Gmail App Passwords: https://myaccount.google.com/apppasswords

Core libraries: Flask, IMAPClient, openpyxl, Gunicorn (see requirements.txt).